ЛАБОРАТОРИЙН АЖЛЫН ЗААВАР

Сэдэв: Deadlock-ыг C хэл дээр симуляц хийх, шинжлэх, сайжруулах

Хичээл	Үйлдлийн систем
Сэдэвчилсэн бүлэг	Chapter 7: Deadlocks
Програмчлалын хэл	C (POSIX threads / pthread), C++

1. Зорилго

Энэхүү лабораторийн ажлын зорилго нь deadlock-ийн онолын ойлголтыг C хэл дээр бичсэн олон урсгалт програмын жишээн дээр ажиглах, өгөгдсөн кодын ажиллагааг шинжлэх, deadlock үүсэх шалтгааныг тодорхойлох, мөн lock авах дарааллыг өөрчлөх замаар deadlock-оос сэргийлсэн хувилбар боловсруулахад оршино.

2. Суралцах үр дүн

- lock, mutex, critical section гэсэн ойлголтуудыг тайлбарлах;
- deadlock үүсэх 4 нөхцөлийг бодит кодтой холбон тайлбарлах;
- pthread ашиглан хоёр thread бүхий програмыг компайл хийж ажиллуулах;
- deadlock үүсгэдэг кодыг ажиглаж, хаанаас гацаж байгааг тайлбарлах;
- lock ordering ашиглан deadlock-гүй хувилбар боловсруулах;
- туршилтын үр дүнг харьцуулж, дүгнэлт гаргах.

3. Онолын товч ойлголт

Deadlock гэдэг нь хоёр буюу түүнээс олон thread эсвэл процесс бие биенийхээ эзэмшиж буй нөөцийг хүлээсээр, хэзээ ч үргэлжлүүлэн ажиллаж чадахгүй болсон төлөв юм. Энэ лабораторийн ажилд нөөц нь mutex lock хэлбэрээр илэрнэ.

Deadlock үүсэх 4 зайлшгүй нөхцөл:

Нөхцөл	Тайлбар
Mutual Exclusion	Нөөцийг нэгэн зэрэг зөвхөн нэг thread эзэмшинэ.
Hold and Wait	Thread нэг lock-ийг барьсан атлаа өөр lock-ийг хүлээнэ.
No Preemption	Авсан lock-ийг бусад thread хүчээр булаан авч чадахгүй.
Circular Wait	Thread-үүд тойрог хүлээлт үүсгэнэ.

4. Хэрэглэгдэх орчин

- OS: Linux эсвэл Windows Subsystem for Linux (WSL) зөвлөж болно;
- Compiler: gcc;
- Library: pthread;
- Editor: VS Code, Code::Blocks, CLion, эсвэл дурын C editor;
- Terminal ашиглан компайл болон ажиллуулалт хийнэ.

5. Жишээ C код: Deadlock үүсгэж болох хувилбар

Доорх жишээ нь хоёр thread хоёр өөр дарааллаар lock авч байгаагаас deadlock үүсэж болохыг харуулна. Оюутан эхлээд энэ кодыг ажиллуулж, дараа нь шинжилгээ хийнэ.

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t lock1;
pthread_mutex_t lock2;

void* thread1_func(void* arg) {
    printf("Thread 1: lock1 авах гэж байна...\n");
    pthread_mutex_lock(&lock1);
    printf("Thread 1: lock1 авлаа\n");

    sleep(1);

    printf("Thread 1: lock2 авах гэж байна...\n");
    pthread_mutex_lock(&lock2);
    printf("Thread 1: lock2 авлаа\n");

    pthread_mutex_unlock(&lock2);
    pthread_mutex_unlock(&lock1);
    printf("Thread 1: ажил дууслаа\n");
    return NULL;
}

void* thread2_func(void* arg) {
    printf("Thread 2: lock2 авах гэж байна...\n");
    pthread_mutex_lock(&lock2);
    printf("Thread 2: lock2 авлаа\n");

    sleep(1);

    printf("Thread 2: lock1 авах гэж байна...\n");
    pthread_mutex_lock(&lock1);
    printf("Thread 2: lock1 авлаа\n");

    pthread_mutex_unlock(&lock1);
    pthread_mutex_unlock(&lock2);
    printf("Thread 2: ажил дууслаа\n");
    return NULL;
}

int main() {
    pthread_t t1, t2;

    pthread_mutex_init(&lock1, NULL);
    pthread_mutex_init(&lock2, NULL);

    pthread_create(&t1, NULL, thread1_func, NULL);
    pthread_create(&t2, NULL, thread2_func, NULL);
```

```
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_mutex_destroy(&lock1);
    pthread_mutex_destroy(&lock2);

    printf("Main: програм дууслаа\n");
    return 0;
}
```

Кодын логик тайлбар

- thread1_func() эхлээд lock1-ийг авч, дараа нь lock2-ийг авах гэж оролдоно.
- thread2_func() эхлээд lock2-ийг авч, дараа нь lock1-ийг авах гэж оролдоно.
- sleep(1) нь хоёр thread-д эхний lock-оо аваад нөгөө lock дээр зэрэг шахуу очих боломж бүрдүүлдэг.
- Хэрэв Thread 1 lock1-ийг, Thread 2 lock2-ийг авсан байвал хоёулаа хоёр дахь lock дээрээ хүлээлтэд орж deadlock үүсэж болно.

6. Компайл хийх ба ажиллуулах заавар

Файлын нэрийг deadlock.c гэж хадгалсан гэж үзвэл дараах командаар компайл хийнэ:

```
gcc deadlock.c -o deadlock -pthread
```

Дараа нь програмыг ажиллуулна:

```
./deadlock
```

Хэрэв deadlock үүсвэл програм төгсгөл хүртлээ явахгүй, тодорхой мөрүүд дээр консоль гаралт зогсоно.

7. Даалгавар

1. Жишээ кодыг уншиж, глобал хувьсагч, mutex lock, thread функц, main() функцийн үүргийг тайлбарлах.

2. Програмыг компайл хийж ажиллуулаад ямар дарааллаар мессежүүд гарч байгааг тэмдэглэх.

3. Програм хаанаас гацаж байгааг тодорхойлж, deadlock яагаад үүсэж байгааг тайлбарлах.

4. Deadlock-ийн 4 нөхцөлийг өгөгдсөн код дээр жишээлэн тайлбарлах.

5. Кодыг өөрчилж хоёр thread хоёулаа lock1 → lock2 гэсэн ижил дарааллаар авах хувилбар боловсруулах.

6. Хоёр хувилбарын үр дүнг харьцуулж, deadlock яагаад арилсныг дүгнэх.

7. Нэмэлтээр trylock, timeout төстэй логик, эсвэл 3 дахь lock ашигласан өргөтгөлөөс дор хаяж нэгийг хийх.

8. Сайжруулсан жишээ код: Deadlock-оос сэргийлсэн хувилбар

Доорх хувилбарт хоёр thread хоёулаа lock-уудыг ижил дарааллаар авч байна. Иймд circular wait нөхцөл арилж, deadlock үүсэхгүй.

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
```

```
pthread_mutex_t lock1;
pthread_mutex_t lock2;

void* safe_thread_func(void* arg) {
    char* name = (char*)arg;

    printf("%s: lock1 авах гэж байна...\n", name);
    pthread_mutex_lock(&lock1);
    printf("%s: lock1 авлаа\n", name);

    sleep(1);

    printf("%s: lock2 авах гэж байна...\n", name);
    pthread_mutex_lock(&lock2);
    printf("%s: lock2 авлаа\n", name);

    printf("%s: critical section-д ажиллаж байна...\n", name);

    pthread_mutex_unlock(&lock2);
    pthread_mutex_unlock(&lock1);

    printf("%s: ажил дууслаа\n", name);
    return NULL;
}

int main() {
    pthread_t t1, t2;

    pthread_mutex_init(&lock1, NULL);
    pthread_mutex_init(&lock2, NULL);

    pthread_create(&t1, NULL, safe_thread_func, "Thread 1");
    pthread_create(&t2, NULL, safe_thread_func, "Thread 2");

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_mutex_destroy(&lock1);
    pthread_mutex_destroy(&lock2);

    printf("Main: deadlock үүсгэлгүй програм дууслаа\n");
    return 0;
}
```

Энэ хувилбар safe байх шалтгаан

- Хоёр thread хоёулаа lock1-ийг эхэлж, дараа нь lock2-ийг авна.
- Нөөц авах дараалал стандартчлагдсан тул circular wait үүсэхгүй.
- Lock ordering нь deadlock prevention-ийн түгээмэл, ойлгомжтой арга юм.

9. Тайлангийн бүтэц

1. Гарчиг

2. Зорилго

3. Онолын үндэслэл

4. Жишээ кодын тайлбар

5. Туршилтын үр дүн

6. Сайжруулсан кодын тайлбар

7. Харьцуулсан шинжилгээ

8. Дүгнэлт

10. Хэлэлцүүлгийн асуултууд

1. Deadlock болон starvation хоёрын ялгаа юу вэ?

2. sleep(1)-ийг арилгавал deadlock үргэлж үүсэх үү?

3. trylock ашиглах нь deadlock-оос бүрэн сэргийлж чадах уу?

4. 3 thread, 3 lock бүхий илүү төвөгтэй deadlock-ийг хэрхэн дүрслэх вэ?

5. Бодит үйлдлийн системд ямар төрлийн нөөцүүд дээр deadlock түгээмэл тохиолдож болох вэ?

11. Дүгнэлт

Энэхүү лабораторийн ажлаар оюутан deadlock-ийн онолыг зөвхөн тодорхойлолтоор бус, програмын бодит гүйцэтгэл дээр ажиглаж ойлгоно. Жишээ код нь чиглүүлэх суурь болж, харин оюутны гол ажил нь кодыг шинжлэх, deadlock үүсэх шалтгааныг тайлбарлах, мөн сайжруулсан хувилбар боловсруулахад чиглэнэ.